

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs

MODULE III: Software Systems, Operating Systems & Device Drivers

Comprehensive Academic Notes — Units 12 through 15 (Duration: 10 Hours, Weightage: 15 Marks)

Unit 12: Softwares — Classification, Taxonomy & Translators

CONCEPTUAL DEFINITION: SOFTWARE

Software is an organized collection of computer programs, algorithmic procedures, data structures, and associated technical documentation that directs computer hardware on precisely how to execute data acquisition, processing, manipulation, transmission, and storage tasks.

While hardware represents the physical semiconductor circuitry and mechanical packaging, software represents the logical instructions that animate the machine. Software is classified into two overarching domains: **System Software** and **Application Software**.

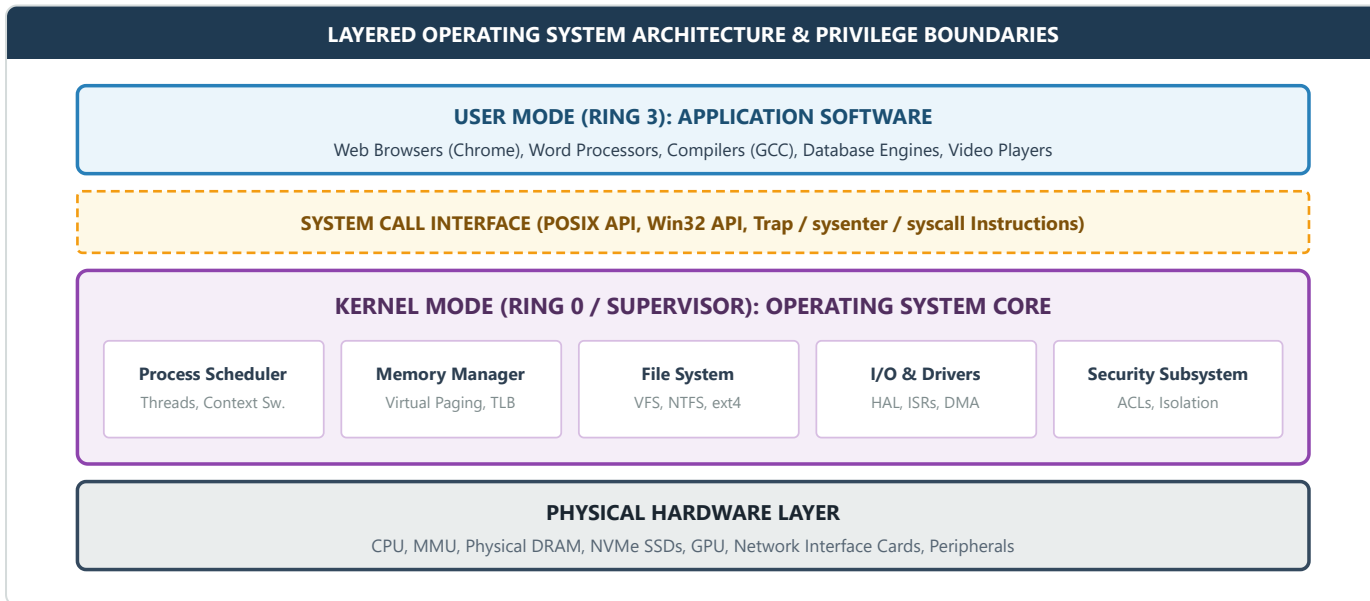


Figure 3.1: Layered Architecture of Computer Software and Operating System Kernel Subsystems.

12.1 System Software

System Software comprises low-level programs that directly manage, control, and interact with the physical computer hardware, providing a stable, abstracted platform upon which application software executes.

1. Operating Systems (OS)

The primary supervisory control program managing hardware resources and presenting an abstract virtual machine interface (e.g., Windows 11, Ubuntu Linux, macOS, FreeBSD).

2. Programming Language Generations & Taxonomy

Programming languages have evolved through five distinct generations (1GL to 5GL), spanning multiple computational paradigms:

- **First-Generation Language (1GL):** Pure binary machine code (**0s and 1s**) directly executed by CPU instruction decoders. Extremely tedious, non-portable, and prone to human error.
- **Second-Generation Language (2GL):** Assembly languages using symbolic mnemonics (e.g., `ADD`, `MOV`, `JMP`, `PUSH`) translated 1-to-1 to binary machine code by an **Assembler**.
- **Third-Generation Language (3GL):** High-level imperative procedural and object-oriented languages (C, C++, Fortran, Java, Pascal) using English keywords and mathematical expressions, translated via Compilers or Interpreters.
- **Fourth-Generation Language (4GL):** Non-procedural, declarative query and data manipulation languages focusing on *what* to achieve rather than *how* to compute (e.g., SQL, MATLAB, SAS).
- **Fifth-Generation Language (5GL):** Constraint-based and logic programming languages used in artificial intelligence and automated theorem proving (e.g., Prolog, Mercury).

3. Programming Paradigms and Typing Systems

- **Imperative / Procedural Paradigm:** Programs are organized around state variables and step-by-step sequences of imperative statements (C, Pascal).
- **Object-Oriented Paradigm (OOP):** Programs model domain entities as encapsulated objects containing private state attributes and public methods (Java, C++, C#). Enforces Encapsulation, Inheritance, and Polymorphism.
- **Functional Paradigm:** Computation is treated as the mathematical evaluation of pure, immutable functions without mutable side effects (Haskell, Lisp, Scala, Erlang).
- **Type Systems:**
 - *Static Typing vs Dynamic Typing:* In statically typed languages (C, Rust, Java), variable data types are checked and verified at compile-time; in dynamically typed languages (Python, JavaScript, Ruby), types are associated with runtime values and checked during execution.
 - *Strong Typing vs Weak Typing:* Strongly typed languages strictly disallow implicit invalid type coercion (e.g., adding an integer to a string raises a runtime error), whereas weakly typed languages perform implicit casting (e.g., C pointer casting, JavaScript string-number concatenation).

4. Memory Management & Garbage Collection

- **Stack Memory vs Heap Memory:**
 - *Stack:* LIFO (Last-In First-Out) memory segment automatically managed by CPU stack pointer register (`RSP`). Stores local variables, return addresses, and function call frames. Extremely fast ($O(1)$ allocation/deallocation), but fixed in size.
-

- *Heap*: Dynamically allocated memory pool managed manually (`malloc()` / `free()` in C, `new` / `delete` in C++) or automatically via a runtime Garbage Collector (GC). Susceptible to memory fragmentation, buffer overflows, dangling pointers, and memory leaks.

- **Automatic Garbage Collection Strategies:**

- *Reference Counting*: Tracks active references to every allocated object; frees memory when count drops to zero. *Limitation*: Fails to reclaim cyclic circular references unless augmented with cycle detection.
- *Tracing Garbage Collection (Mark-and-Sweep)*: Traverses the graph of live object references starting from root pointers (stack frames, global registers). The *Mark phase* sets a bit on reachable objects; the *Sweep phase* traverses the entire heap, reclaiming memory occupied by unmarked unreachable objects.
- *Generational GC*: Exploits the weak generational hypothesis (most objects die shortly after allocation). The heap is partitioned into *Young Generation (Eden + Survivor spaces)* and *Old / Tenured Generation*. Frequent minor collections scavenge the Eden space with minimal pause latency.

5. Language Translators: Compilers, Interpreters & Assemblers

- **Assembler**: Translates assembly mnemonics into object code, resolving symbolic labels into relative memory offsets via two passes (Pass 1 builds Symbol Table, Pass 2 emits machine code).
- **Compiler**: Translates entire high-level source code programs into target machine code or intermediate bytecode in a single batch process before execution. Generates optimized standalone binaries (e.g., GCC, Clang, MSVC).
- **Interpreter**: Reads, parses, and executes high-level source code line-by-line in real time without creating standalone machine binaries (e.g., CPython, standard Ruby). Provides rapid development and dynamic typing at the cost of slower execution speeds.
- **Hybrid / Just-In-Time (JIT) Compilers**: Modern runtimes (Java Virtual Machine JVM, V8 JavaScript engine, .NET CLR) first compile source code into portable intermediate Bytecode, and then dynamically compile hot code loops into native CPU machine instructions at runtime using JIT compilation.

6. Deep Dive: Six Phases of a Modern Compiler

1. **Lexical Analysis (Scanner)**: Scans the input character stream and groups them into lexical units called **Tokens** (e.g., `<KEYWORD, "while">`, `<IDENTIFIER, "counter">`, `<REL_OP, "<=">`, `<NUM, 100>`), while discarding whitespace and comments.
2. **Syntax Analysis (Parser)**: Implements formal Context-Free Grammars (CFG) using algorithms like LL(k), LR(k), or LALR to verify grammatical correctness and construct an **Abstract Syntax Tree (AST)**.
3. **Semantic Analysis**: Verifies type safety, enforces variable scope and declaration rules, checks array bounds constraints, and ensures function parameter compatibility using the Symbol Table.
4. **Intermediate Code Generation (ICG)**: Translates the validated AST into a machine-independent intermediate language, typically **Three-Address Code (TAC)**:

$$t_1 = y \times z, \quad t_2 = x + t_1$$

5. **Code Optimization**: Transforms intermediate code to improve execution speed and reduce memory consumption:
 - *Constant Folding & Propagation*: Pre-calculating compile-time constants (e.g., replacing `3 + 5` with `8`).
 - *Dead Code Elimination*: Removing code segments that can never execute or whose results are never used.
 - *Loop-Invariant Code Motion*: Moving computations outside loops if their results do not change across iterations.

- *Strength Reduction*: Replacing computationally expensive operations with cheaper ones (e.g., replacing $x \times 2$ with $x \ll 1$).
- *Common Subexpression Elimination*: Identifying redundant calculations and caching their results.

6. **Target Code Generation**: Maps intermediate variables to physical CPU registers using Graph Coloring Algorithms and emits native binary machine code instructions.

7. Linkers, Loaders & Binary Formats

- **Static Linking vs Dynamic Linking:**

- *Static Linker*: Copies all required library routines (e.g., `libc.a`) directly into the final executable binary, resulting in larger files that run independently without external dependency issues.
- *Dynamic Linker*: Inserts symbolic references to shared libraries (Dynamic Link Libraries — `.dll` on Windows, Shared Objects — `.so` on Linux). Multiple executing programs share a single copy of library code in RAM, reducing memory footprint. Relocation is handled at runtime via the **Global Offset Table (GOT)** and **Procedure Linkage Table (PLT)** for Position-Independent Code (PIC).

- **Executable File Formats (ELF & PE):**

- *Executable and Linkable Format (ELF - Linux)*: Contains an ELF Header, Program Header Table (segments mapped to memory), Section Header Table (`.text` code, `.data` initialized globals, `.bss` zero-initialized globals, `.rodata` read-only constants, `.symtab` symbol table), and dynamic linking tables.
- *Portable Executable (PE - Windows)*: Extends the MS-DOS MZ header with PE Header, Optional Header, Data Directories (Export/Import tables), and section headers (`.text`, `.rdata`, `.data`, `.pdata`).

- **Loader**: The OS program that reads the executable file header, allocates virtual memory segments (Code/Text, Initialized Data, BSS, Stack, Heap), maps shared libraries, initializes CPU registers, and transfers execution to the program entry point (`main()`).

12.2 Application Software

Application Software encompasses programs engineered to perform end-user tasks, productivity workflows, or business data processing:

- **General-Purpose Software**: Mass-market productivity applications: Word Processors (Microsoft Word, LibreOffice Writer), Spreadsheets (Microsoft Excel), Web Browsers (Google Chrome, Mozilla Firefox), and Media Players (VLC).
- **Specialized / Bespoke Software**: Custom-engineered software tailored to specific domains: Medical PACS DICOM imaging systems, Aircraft Avionics control, Electronic Design Automation (EDA - Cadence, KiCad), and CAD/CAM modeling (AutoCAD, SolidWorks).
- **Enterprise Systems**: Large-scale multi-user business platforms: Enterprise Resource Planning (ERP - SAP S/4HANA), Customer Relationship Management (CRM - Salesforce), and Relational Database Engines (PostgreSQL, Oracle DB).

Comparative Matrix: System Software vs Application Software

Evaluation Parameter	System Software	Application Software
Primary Purpose	Manages, controls, and abstracts physical hardware	Executes specific end-user business or creative tasks
Hardware Access	Direct register-level and physical I/O interaction	Zero direct hardware access (restricted to OS System Calls)
Execution Privilege	Privileged Kernel Mode (Ring 0 / Supervisor)	Unprivileged User Mode (Ring 3)
Operational Dependency	Runs autonomously; provides runtime environment	Completely dependent on an underlying OS to execute
Programming Languages	C, C++, Rust, Assembly	Python, Java, JavaScript, C#, Swift, PHP, Go

Unit 13: Operating Systems — Architecture, Scheduling, POST & Booting

13.1 Fundamental Responsibilities of an Operating System

An **Operating System (OS)** is the master control program that acts as an intermediary between computer hardware and user applications. It fulfills two primary roles:

- **Extended Machine (Virtual Machine):** Provides clean, high-level abstractions (files, sockets, virtual memory, processes) that hide the low-level complexities of raw hardware registers, interrupts, and disk sectors.
- **Resource Allocator & Arbiter:** Manages finite hardware resources (CPU cores, DRAM, storage space, network bandwidth) among competing user processes fairly, efficiently, and securely.

Core Operating System Subsystems:

1. **Process Management & CPU Scheduling:** Creating, scheduling, synchronizing, and terminating processes and threads.
 - **7-State Process Model:** Processes transition through: *New* → *Ready* → *Running* → *Waiting/Blocked* → *Terminated*, along with *Ready-Suspended* and *Blocked-Suspended* when swapped out to disk secondary storage.
 - **Process Control Block (PCB):** The kernel data structure storing process state, PID, Program Counter, CPU registers, memory limits, open file descriptors, and scheduling priority.
2. **Process Synchronization & Deadlocks:**
 - **Critical Section Problem:** Ensures mutual exclusion when multiple concurrent processes access shared memory. A valid solution must satisfy: *Mutual Exclusion*, *Progress*, and *Bounded Waiting*.
 - **Synchronization Primitives:** Semaphores (Counting and Binary), Mutex Locks, Condition Variables, and Spinlocks.

- **Deadlock & Coffman Conditions:** A state where two or more processes are permanently blocked waiting for resources held by each other. Deadlock can occur if and only if all four **Coffman Conditions** hold simultaneously:
 1. *Mutual Exclusion:* Resources cannot be shared simultaneously.
 2. *Hold and Wait:* Processes hold allocated resources while requesting new ones.
 3. *No Preemption:* Resources cannot be forcibly confiscated from a process.
 4. *Circular Wait:* A closed chain of processes exists where each process waits for a resource held by the next.
- **Deadlock Handling:** Banker's Algorithm for deadlock avoidance, Resource Allocation Graphs (RAG) for detection, and process termination/preemption for recovery.

3. **Memory Management & Virtual Paging:** Providing each process with an isolated, contiguous virtual address space mapped to physical DRAM via multi-level Page Tables and the hardware Memory Management Unit (MMU).
4. **File System & Secondary Storage:** Structuring persistent disk blocks into hierarchical directories, managing metadata inodes, and enforcing access control lists (ACLs).
5. **I/O Subsystem:** Providing uniform driver interfaces, buffering, caching, spooling, and DMA coordination.

WORKED EXAMPLE 13.1: CPU SCHEDULING COMPARISON

Problem: Four processes arrive at time $t = 0$ with burst times: $P_1 = 6$ ms, $P_2 = 8$ ms, $P_3 = 7$ ms, $P_4 = 3$ ms. Calculate Average Waiting Time under FCFS and Shortest Job First (SJF).

1. First-Come First-Served (FCFS):

Gantt Chart: | P_1 (0–6) | P_2 (6–14) | P_3 (14–21) | P_4 (21–24) |

- $WT(P_1) = 0$ ms, $WT(P_2) = 6$ ms, $WT(P_3) = 14$ ms, $WT(P_4) = 21$ ms
- **Average Waiting Time (FCFS):** $\frac{0+6+14+21}{4} = \frac{41}{4} = 10.25$ ms.

2. Shortest Job First (SJF - Non-preemptive):

Execution Order: $P_4(3\text{ms}) \rightarrow P_1(6\text{ms}) \rightarrow P_3(7\text{ms}) \rightarrow P_2(8\text{ms})$

Gantt Chart: | P_4 (0–3) | P_1 (3–9) | P_3 (9–16) | P_2 (16–24) |

- $WT(P_4) = 0$ ms, $WT(P_1) = 3$ ms, $WT(P_3) = 9$ ms, $WT(P_2) = 16$ ms
- **Average Waiting Time (SJF):** $\frac{0+3+9+16}{4} = \frac{28}{4} = 7.00$ ms (Optimal).

WORKED EXAMPLE 13.2: BANKER'S ALGORITHM SAFETY EVALUATION

Consider 5 processes (P_0, P_1, P_2, P_3, P_4) and 3 resource types ($A = 10, B = 5, C = 7$).

Currently allocated resources:

$$P_0 : (0, 1, 0), \quad P_1 : (2, 0, 0), \quad P_2 : (3, 0, 2), \quad P_3 : (2, 1, 1), \quad P_4 : (0, 0, 2)$$

$$\text{Total Allocated: } A = 7, B = 2, C = 5 \implies \text{Available: } A = 3, B = 3, C = 2$$

Maximum Demand Matrix (Max):

$$P_0 : (7, 5, 3), \quad P_1 : (3, 2, 2), \quad P_2 : (9, 0, 2), \quad P_3 : (2, 2, 2), \quad P_4 : (4, 3, 3)$$

Remaining Need Matrix (**Need** = **Max** – **Allocation**):

$$P_0 : (7, 4, 3), \quad P_1 : (1, 2, 2), \quad P_2 : (6, 0, 0), \quad P_3 : (0, 1, 1), \quad P_4 : (4, 3, 1)$$

Safety Algorithm Execution:

1. Check P_1 : $\text{Need}(1, 2, 2) \leq \text{Available}(3, 3, 2) \implies \text{Execute } P_1$. New Available = $(3, 3, 2) + (2, 0, 0) = (5, 3, 2)$.
2. Check P_3 : $\text{Need}(0, 1, 1) \leq \text{Available}(5, 3, 2) \implies \text{Execute } P_3$. New Available = $(5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.
3. Check P_0 : $\text{Need}(7, 4, 3) \leq \text{Available}(7, 4, 3) \implies \text{Execute } P_0$. New Available = $(7, 4, 3) + (0, 1, 0) = (7, 5, 3)$.
4. Check P_2 : $\text{Need}(6, 0, 0) \leq \text{Available}(7, 5, 3) \implies \text{Execute } P_2$. New Available = $(7, 5, 3) + (3, 0, 2) = (10, 5, 5)$.
5. Check P_4 : $\text{Need}(4, 3, 1) \leq \text{Available}(10, 5, 5) \implies \text{Execute } P_4$. New Available = $(10, 5, 5) + (0, 0, 2) = (10, 5, 7)$.

Conclusion: The system is in a provably **Safe State** with safe sequence $\langle P_1, P_3, P_0, P_2, P_4 \rangle$. No deadlock can occur.

WORKED EXAMPLE 13.3: VIRTUAL MEMORY PAGE REPLACEMENT COMPARISON

Problem: Given a page reference string: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3** and **3** physical memory page frames (initially empty). Compute the total page faults under **FIFO** and **LRU (Least Recently Used)**.

1. First-In First-Out (FIFO) Replacement:

Ref	Frame 1	Frame 2	Frame 3	Status / Fault
7	7	-	-	Page Fault (1)
0	7	0	-	Page Fault (2)
1	7	0	1	Page Fault (3)
2	2	0	1	Page Fault (4) — Replaces 7 (oldest)
0	2	0	1	Hit
3	2	3	1	Page Fault (5) — Replaces 0
0	2	3	0	Page Fault (6) — Replaces 1
4	4	3	0	Page Fault (7) — Replaces 2
2	4	2	0	Page Fault (8) — Replaces 3
3	4	2	3	Page Fault (9) — Replaces 0

Total FIFO Page Faults: 9 Faults (Suffers from *Belady's Anomaly* where adding frames can increase faults).

2. Least Recently Used (LRU) Replacement:

Ref	Frame 1	Frame 2	Frame 3	Status / Fault
7	7	-	-	Page Fault (1)
0	7	0	-	Page Fault (2)
1	7	0	1	Page Fault (3)
2	2	0	1	Page Fault (4) — Replaces 7 (least recently used)
0	2	0	1	Hit (0 becomes most recent)
3	2	0	3	Page Fault (5) — Replaces 1
0	2	0	3	Hit
4	4	0	3	Page Fault (6) — Replaces 2
2	4	0	2	Page Fault (7) — Replaces 3
3	4	3	2	Page Fault (8) — Replaces 0

Total LRU Page Faults: 8 Faults (LRU belongs to the stack algorithm class and never exhibits Belady's Anomaly).

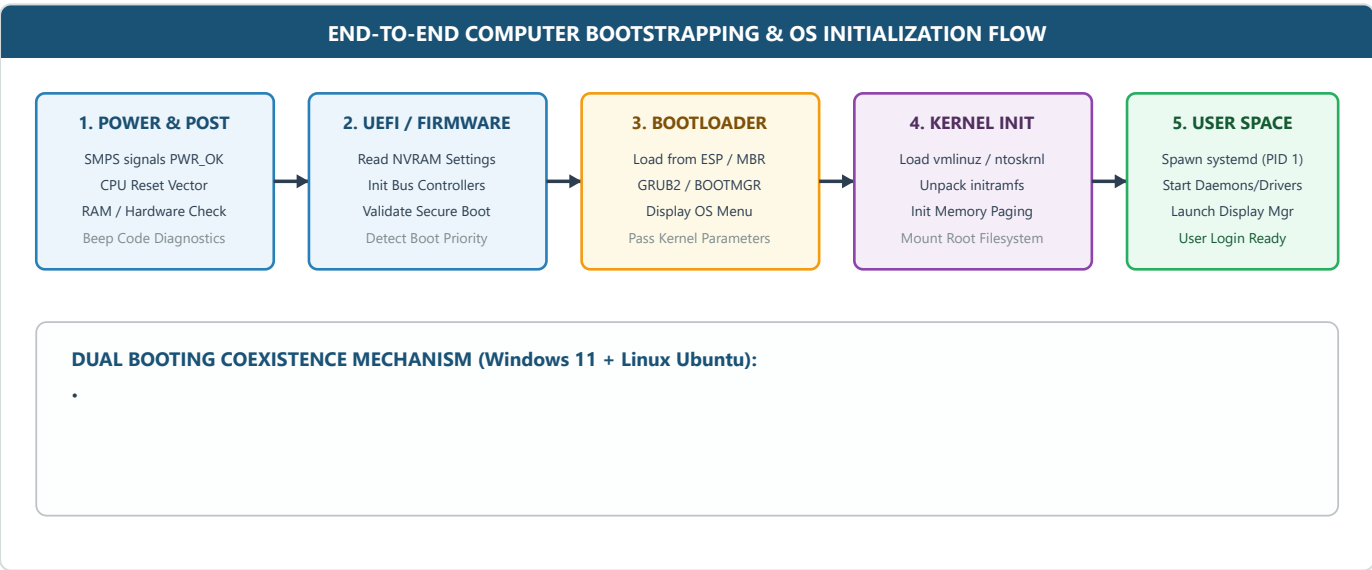
13.2 Operating System Classifications

- **Batch Operating Systems:** Early non-interactive systems where jobs with similar requirements were batched together onto magnetic media and executed sequentially by an operator (e.g., IBM OS/360).
- **Multiprogramming Systems:** Increases CPU utilization by keeping multiple jobs in memory simultaneously. When the active job initiates an I/O wait, the OS switches the CPU to another ready job.
- **Time-Sharing (Multitasking) Systems:** Allocates CPU execution time into millisecond time slices (quantums) across multiple active processes via Round Robin scheduling, providing interactive responsiveness for multiple concurrent users.
- **Multiprocessing Systems:** Utilizes two or more physical CPUs or multi-core processors sharing common memory (Symmetric Multiprocessing — SMP) to achieve true parallel execution.
- **Distributed Systems:** Manages a network of physically separated computational nodes communicating over networks, appearing to end-users as a single unified computing facility.
- **Real-Time Operating Systems (RTOS):** Engineered for applications where correctness depends not only on logical output but on meeting strict deterministic timing deadlines:
 - **Hard RTOS:** Missing a single deadline causes total system failure (e.g., pacemaker control, automotive airbag deployment, flight avionics).
 - **Soft RTOS:** Missing a deadline degrades quality of service without system failure (e.g., video streaming, audio playback).

13.3 Kernel Architectures: Monolithic vs Microkernel vs Hybrid

- **Monolithic Kernel (e.g., Linux, FreeBSD):** Entire operating system core — Process Scheduler, Virtual Memory, File Systems, Network Stacks, and all Device Drivers — executes inside a single shared, highly privileged address space in **Ring 0**.
 - *Advantages:* Maximum performance; inter-module communication occurs via ultra-fast direct C function calls.
 - *Disadvantages:* Large kernel codebase; a bug or memory corruption in any third-party driver can crash the entire system.
- **Microkernel (e.g., Minix, QNX, seL4):** Only the absolute bare essentials (Thread Scheduling, Low-level IPC, Address Space translation) run in Ring 0. All other services (File System, Networking, Device Drivers) run as isolated user-mode server processes in **Ring 3**.
 - *Advantages:* High reliability, modularity, and fault tolerance; crashing drivers do not crash the system.
 - *Disadvantages:* Performance overhead caused by frequent user/kernel context switches and message-passing IPC.
- **Hybrid Kernel (e.g., Windows NT, Apple macOS XNU):** Combines monolithic speed with microkernel modularity, running subsystems in kernel space while maintaining modular driver interfaces.

13.4 Power-On Self-Test (POST) & Bootstrapping Sequence



EFI System Partition (ESP): Formatted as FAT32 (100--512 MB), houses \EFI\Microsoft\Boot\bootmgfw.efi and \EFI\ubuntu\grubx64.efi. • **GRUB2 Boot Manager:** Scans storage partitions via `os-prober`, detects Windows BCD, and presents user with interactive OS selection. • **Critical Setting:** Disable Windows "Fast Startup" in Control Panel to prevent NTFS partition dirty-bit locking and data corruption.

Figure 3.2: Complete System Bootstrapping Sequence and Dual-Boot Architecture.

Detailed Execution Steps of System Bootstrapping:

1. **Power Sequencing & Reset Vector:** The SMPS asserts `PWR_OK`. The CPU resets its registers and jumps to the hardcoded Reset Vector (x86: physical address `0xFFFFFFFF`), executing the initial jump to UEFI Flash ROM code.

- 2. **Power-On Self-Test (POST):** Low-level firmware routines test CPU arithmetic logic, ROM checksums, RTC/NVRAM clocks, DRAM data lines, and enumerate PCIe devices. Hardware errors are signaled via piezoelectric beep codes or motherboard 2-digit 7-segment hex displays (Port 80h).
- 3. **Firmware Initialization & Boot Selection:** UEFI initializes bus controllers, validates Secure Boot digital signatures, reads boot priority from NVRAM, and loads the target EFI bootloader from the EFI System Partition (ESP).
- 4. **Bootloader Handover:** Windows `bootmgfw.efi` or Linux `grubx64.efi` executes, loads kernel binary images into RAM (`ntoskrnl.exe` or `vmlinuz`), and passes hardware configuration parameters.
- 5. **Kernel Initialization:** The kernel initializes memory management paging, unpacks the initial RAM disk (`initramfs`), mounts the root filesystem, initializes built-in drivers, and spawns the first user-space initialization process (`systemd` on Linux with PID 1 / `smss.exe` on Windows).

Unit 14: OS Installation, Partitioning, Boot Managers & File Systems

14.1 Bootable Media Creation & Installation Workflow

Operating system installation begins with creating bootable installation media (typically a USB flash drive).

- **ISO Image Architecture:** An uncompressed optical disc sector image archive (ISO 9660 / UDF format) containing the entire OS installer, compressed kernel squashfs images, package repositories, and bootable El Torito boot records.
- **Flashing Utilities (Rufus, Balena Etcher, Ventoy):** Software tools that unpack the ISO and write raw sector boot blocks onto the USB drive. When using Rufus, users configure target partition scheme (MBR or GPT) and target system firmware (BIOS or UEFI).

14.2 Disk Partitioning: MBR vs GPT

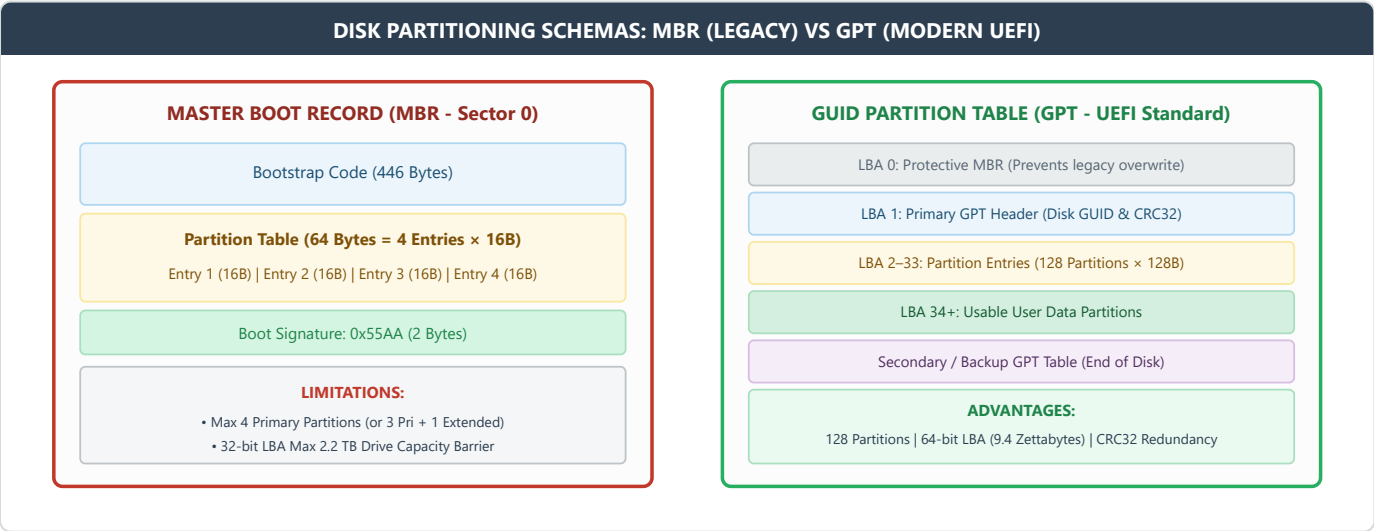


Figure 3.3: Architectural Layout of MBR Partition Table vs Modern GPT Schema.

1. Master Boot Record (MBR - Legacy Schema)

Resides at Sector 0 (exactly 512 bytes):

- **Bootstrap Code (446 Bytes):** Contains the Stage-1 machine code executed by BIOS to scan the partition table, find the partition flagged as active/bootable (`0x80`), and load its Volume Boot Record (VBR).
- **Partition Table (64 Bytes):** Exactly four 16-byte partition records. Each record stores:
 - Byte 0: Boot indicator (`0x80` = Active bootable, `0x00` = Inactive).
 - Bytes 1–3: Starting Cylinder-Head-Sector (CHS) coordinates.
 - Byte 4: Partition Type ID (e.g., `0x07` for NTFS, `0x83` for Linux native).
 - Bytes 5–7: Ending CHS coordinates.
 - Bytes 8–11: Starting Logical Block Address (LBA) sector number (32-bit integer).
 - Bytes 12–15: Total sector count in the partition (32-bit integer).
- **Boot Signature (2 Bytes):** Hex word `0x55AA` verifying sector validity.
- **The 2.2 TB Capacity Barrier:** MBR uses 32-bit Logical Block Addressing (LBA) sector numbers. With standard 512-byte sectors:

$$\text{Max MBR Size} = 2^{32} \times 512 \text{ bytes} = 2,199,023,255,552 \text{ bytes} \approx \mathbf{2.19 \text{ TB}}$$

Any storage capacity exceeding **2.2 TB** on an MBR disk is completely unaddressable.

2. GUID Partition Table (GPT - Modern UEFI Schema)

Developed as part of the Unified EFI standard, GPT replaces legacy MBR:

- **64-Bit LBA Addressing:** Supports drive capacities up to $2^{64} \times 512 \text{ bytes} \approx \mathbf{9.4 \text{ Zettabytes}}$ ($9.4 \times 10^{21} \text{ bytes}$).
- **Partition Count:** Supports a minimum of **128 distinct primary partitions** without needing extended/logical partition hacks.
- **Data Redundancy & Integrity:** GPT stores a Primary Header and Partition Table at the beginning of the disk (LBA 1–33) and a **Secondary Backup GPT Header** at the trailing end of the disk. Every partition entry and header is protected by **CRC32 Checksums**; if corruption occurs, UEFI automatically restores the partition table from the backup copy.
- **Protective MBR:** GPT places a dummy MBR at LBA 0 with a single partition type `0xEE` spanning the entire drive, preventing legacy MBR-only utilities from misidentifying the disk as unpartitioned and destroying user data.

14.3 Boot Managers: Windows BOOTMGR vs GNU GRUB2

- **Windows Boot Manager (`BOOTMGR`):** The proprietary bootloader for Windows. Reads configuration data from the **Boot Configuration Data (BCD)** registry hive, verifies driver digital signatures, loads `winload.efi` (or `winload.exe`), initializes the Windows Kernel (`ntoskrnl.exe`), and transfers execution.
- **GNU GRand Unified Bootloader (GRUB2):** The dominant open-source multi-boot loader in Linux environments:
 - **Stage 1 (`boot.img`):** Tiny 512-byte code residing in MBR sector 0 or loaded by UEFI.
 - **Stage 1.5 / Core (`core.img`):** Contains file system drivers (ext4, NTFS, FAT32) allowing GRUB to access Stage 2 files.
 - **Stage 2 (`/boot/grub/grub.cfg`):** Displays the interactive multi-boot menu, allows runtime editing of kernel boot parameters, unpacks the `initramfs` RAM disk, loads the compressed Linux kernel (`vmlinuz`), and passes system control.

14.4 Dual Booting Architecture (Windows & Linux)

Dual booting enables a single physical computer to host multiple independent operating systems selectable via a boot menu at startup.

- **Partition Layout:**
 - **EFI System Partition (ESP):** Formatted strictly as **FAT32 (100–512 MB)**. Contains `\EFI\Microsoft\Boot\bootmgfw.efi` and `\EFI\ubuntu\grubx64.efi` simultaneously.
 - **Windows C: Partition:** Formatted as NTFS (100+ GB).
 - **Linux Root Partition (/):** Formatted as ext4 (50+ GB).
 - **Linux Swap Partition:** Dedicated virtual memory paging space.
- **Chainloading Mechanism:** During UEFI boot, GRUB is executed as the primary boot manager. GRUB executes its `os-prober` script, discovers the Windows BCD entry on the ESP, and offers an interactive choice. Selecting Windows "chainloads" execution directly to `bootmgfw.efi` .
- **Windows Fast Startup Caveat:** Windows "Fast Startup" puts the OS into a hybrid hibernation state that locks the NTFS file system metadata dirty-bit. If Linux mounts that NTFS partition while Fast Startup is active, severe disk corruption occurs. Fast Startup **must be disabled** in dual-boot setups.

14.5 File Systems: FAT vs NTFS vs ext4

Feature / Metric	FAT32	exFAT	NTFS	ext4
Primary OS	Legacy Windows / Flash	Universal Flash Media	Modern Windows (10/11)	Linux Distributions
Max Single File Size	4 GB – 1 Byte	16 Exabytes	16 Terabytes	16 Terabytes
Max Volume Size	2 TB to 16 TB	128 Petabytes	8 Petabytes	1 Exabyte
Journaling Support	No (Prone to corruption)	No	Yes (USN Journal)	Yes (Superblock + Extents)
File Permissions & ACLs	None	None	Granular Windows ACLs	POSIX (rwx, User/Group/Other)
Compression / Encryption	No	No	Native NTFS Compression & EFS	Native transparent encryption

Internal File System Architectures & Journaling Modes:

- **FAT32 Cluster Chains:** Files are stored as linked lists of clusters in the File Allocation Table (FAT). Each entry contains a 28-bit cluster index pointing to the next cluster, with `0x0FFFFFFF` indicating End-Of-File (EOF). Because file access requires sequentially traversing the FAT chain, random access on fragmented drives suffers significant performance degradation.

- **NTFS (Master File Table - MFT):** Everything on an NTFS volume is a file, including metadata files (e.g., `$MFT`, `$LogFile`, `$Bitmap`). Each file has a 1024-byte record in the MFT containing resident data or non-resident data run-lists. Directories are indexed using balanced B-Trees for rapid $O(\log n)$ file lookups.
- **ext4 (Inodes & Extent Trees):** An ext4 filesystem divides the disk into **Block Groups**. Each file is represented by an **Inode** containing file metadata (size, timestamps, POSIX permissions, owner UID/GID) and an **Extent Tree** (a contiguous run of physical blocks represented by starting block and length), replacing legacy indirect block pointers to drastically reduce disk fragmentation. ext4 supports three journaling modes:
 - *Journal:* Both metadata and file data are written to the journal before committing to disk (highest integrity, lowest write speed).
 - *Ordered:* Only metadata is journaled, but file data blocks are flushed to disk before committing metadata (default Linux mode).
 - *Writeback:* Metadata is journaled without ordering guarantees for data blocks (highest performance, risk of stale data on crash).

Unit 15: Device Drivers & Kernel Subsystems

15.1 Need and Role of Device Drivers

A **Device Driver** is a specialized system software program that serves as a translator and abstraction bridge between the generic I/O subsystem of the Operating System kernel and the specific electrical, register-level hardware mechanics of physical peripheral devices.

WHY DEVICE DRIVERS ARE INDISPENSABLE

Hardware manufacturers build peripherals (Realtek audio chips, Intel Wi-Fi cards, NVIDIA GPUs, HP printers) with wildly differing internal register layouts, command timing sequences, and physical firmware interfaces. The OS kernel cannot include hardcoded machine logic for millions of peripheral models. Instead, the OS defines standardized abstract I/O interfaces (e.g., `read()`, `write()`, `open()`, `close()`, `ioctl()`). The device driver implements these standardized calls by issuing device-specific commands to hardware registers.

15.2 Protection Rings & Driver Execution Modes

Modern x86/x64 processors implement hardware-enforced **Hierarchical Protection Rings** to isolate software components:

- **Ring 0 (Supervisor / Kernel Mode):** Highest privilege level. Code in Ring 0 has unrestricted direct access to all CPU instructions (including privileged instructions like `CLI`, `STI`, `IN`, `OUT`, `LIDT`), all physical memory addresses, and all I/O hardware ports.
 - **Kernel-Mode Device Drivers:** Most high-performance drivers (GPU display drivers, storage controllers, network cards) execute in Ring 0. *Risk:* A single null-pointer dereference or unhandled exception in a Kernel-Mode driver immediately crashes the entire operating system, resulting in a **Blue Screen of Death (BSOD)** in Windows or a **Kernel Panic** in Linux.

- **Ring 3 (User Mode):** Lowest privilege level. Standard user applications execute here with restricted virtual memory access. Direct hardware I/O instructions are strictly prohibited; any unauthorized attempt raises a hardware General Protection Fault (GPF).
 - **User-Mode Driver Framework (UMDF):** Modern OS designs execute non-time-critical drivers (software printers, USB sensors, smart card readers) in Ring 3 isolated worker processes. If a User-Mode driver crashes, the OS simply restarts that specific process without affecting system stability.

15.3 Driver-Hardware Interaction Mechanisms

Device drivers coordinate hardware communication through four foundational mechanisms:

1. **Port-Mapped I/O (PMIO):** The CPU maintains a dedicated 16-bit physical I/O port address space separate from system RAM, accessed via specialized assembly instructions: `IN AL, DX` (read byte from port in `DX`) and `OUT DX, AL` (write byte to port). Legacy x86 architecture used by legacy keyboards (Port 60h/64h), floppy disk controllers (Port 3F0h), and serial COM ports.
2. **Memory-Mapped I/O (MMIO):** Modern high-speed standard where peripheral device control registers and internal buffers are mapped directly into the physical memory address space of the CPU. The driver reads and writes to device hardware using standard memory instructions (e.g., `MOV`), benefiting from CPU memory caching, speculative execution pipelines, and high-speed bus interconnects.
3. **Interrupts, APIC, MSI-X & ISR Architecture:**
 - *Legacy PIC vs Modern APIC:* Legacy systems routed 15 IRQ lines through two cascaded Intel 8259A Programmable Interrupt Controllers. Modern multi-core processors utilize the **Advanced Programmable Interrupt Controller (APIC)** architecture, consisting of a *Local APIC (LAPIC)* per CPU core and an *I/O APIC* on the motherboard chipset, supporting up to 240 interrupt vectors per core with inter-processor interrupts (IPI).
 - *Message Signaled Interrupts (MSI / MSI-X):* PCIe devices eliminate dedicated physical interrupt sideband wires by writing special 32-bit/64-bit address data packets directly to the host memory controller, triggering CPU interrupts. **MSI-X** supports up to 2,048 individual interrupt vectors per device, allowing multi-queue NVMe drives and 100GbE NICs to bind separate hardware interrupt queues directly to individual CPU cores for zero-contention parallel processing.
 - *Top-Half vs Bottom-Half Processing:* To prevent stalling the CPU, modern drivers divide interrupt handling: the *Top-Half (ISR)* runs immediately with hardware interrupts disabled to acknowledge the peripheral, clear hardware status bits, and copy packet/block buffers into RAM; the *Bottom-Half (Deferred Procedure Call DPC in Windows / Tasklets & Workqueues in Linux)* processes the bulk data asynchronously at lower interrupt request levels (IRQL) with hardware interrupts re-enabled.
4. **Direct Memory Access (DMA) & Scatter-Gather Descriptors:**
 - Without DMA, the CPU would be forced to read every incoming data byte from hardware into a CPU register and then write it to RAM (Programmed I/O - PIO), wasting 100% of CPU cycles and throttling system throughput.
 - A **DMA Controller (DMAC)** allows high-speed peripherals (NVMe SSDs, Gigabit NICs) to transfer massive data blocks directly into and out of system RAM over PCIe/system buses without CPU intervention. The driver initializes physical source/destination addresses and byte length, and the DMAC transfers data autonomously at memory bus speeds.
 - **Scatter-Gather DMA:** Real-world operating systems allocate virtual memory in non-contiguous physical 4 KB pages. Standard DMA would require expensive memory buffer copies into contiguous bounce buffers. With *Scatter-Gather DMA*, the driver constructs a linked list of physical memory descriptors (called a **Scatter-Gather List - SGL**), where each descriptor contains a 64-bit physical memory pointer and byte

length. The hardware DMAC traverses the descriptor chain automatically, reading from or writing to discontinuous physical RAM pages in a single uninterrupted hardware transaction.

15.4 Linux Kernel Module (LKM) & Windows Driver Framework (WDF)

- **Linux Loadable Kernel Modules (LKM):** LKMs are dynamically linked into the running kernel without rebooting using user-space utilities (`insmod` , `modprobe` , `rmmmod`). LKMs provide standard entry/exit points (`module_init()` and `module_exit()`) and interface with user-space via Character (`/dev/chr`), Block (`/dev/sda`), or Network (`eth0`) abstractions.

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Computer Science Faculty");
MODULE_DESCRIPTION("Sample Linux Character Device Driver");

static int device_open(struct inode *inode, struct file *file) {
    pr_info("Device opened successfully\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = device_open,
};

static int __init sample_driver_init(void) {
    pr_info("Registering character device driver\n");
    return register_chrdev(240, "sample_dev", &fops);
}

static void __exit sample_driver_exit(void) {
    pr_info("Unregistering character device driver\n");
    unregister_chrdev(240, "sample_dev");
}

module_init(sample_driver_init);
module_exit(sample_driver_exit);
```

- **Windows Driver Framework (WDF):** Modern unified driver architecture in Windows:
 - *Kernel-Mode Driver Framework (KMDF):* Object-oriented C-based framework handling I/O Queuing, Plug and Play (PnP), and Power Management state machines for Ring 0 drivers. Handles I/O Request Packets (IRPs) through structured dispatch tables.
 - *User-Mode Driver Framework (UMDF):* COM-based C++ framework running drivers inside isolated user-mode Host Processes (`WUDFHost.exe`). If a user-mode driver encounters an unhandled exception, the OS restarts the worker process without destabilizing the Windows kernel.